

OS ORANGE

PES-VCS: Building a Version Control System from Scratch

Name: SOUMIK SINHA

SRN: PES1UG24CS463

Section: 4-H

Phase 1: Object Storage

Implementation

In this phase I had to implement two functions in `object.c` — `object_write` and `object_read`. The whole point of this is that objects are stored by the hash of their contents, not by filename. So if two files have the same content they end up pointing to the same stored object, which saves space.

For `object_write` I first build a header string — something like "blob 16" followed by a null byte, then the actual file data. The SHA-256 hash is computed over this combined buffer. The first 2 characters of the hex hash become the subdirectory under `.pes/objects/` and the rest becomes the filename. This sharding is just so we don't end up with thousands of files in one directory.

I also used the temp-file-then-rename trick for writing. The file first gets written to a temp path, then renamed to the final location. On Linux, `rename()` is atomic so even if the process crashes mid-write the object store doesn't end up with corrupted half-written files.

`object_read` does the reverse — reads the file, recomputes SHA-256 and checks it against the filename to make sure nothing got corrupted. Then parses the header to get the type and size, and returns only the data after the null byte.

Screenshots

1A: `./test_objects` passing

```
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
error: object integrity check failed
PASS: integrity check

All Phase 1 tests passed.
```

1B: `find .pes/objects -type f` showing sharded structure

```
(kali@kali)~[~/PES1UG24CS463-pes-vcs]
$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2: Tree Objects

Implementation

This phase was about building the tree object which represents a directory snapshot. The function `tree_from_index` takes the flat list of staged files and organises them into the nested tree structure.

The main issue I ran into was handling paths with slashes in them, like `src/main.c`. You can't just put everything in one root tree — you have to group files by their parent directory and build subtrees recursively. So I loop through the index entries, separate the top-level files from the ones inside subdirectories, then for each unique subdirectory I collect its entries, strip the directory prefix from their paths, and recursively call `tree_from_index` on that subset.

I kept a `seen_dirs` array to make sure I don't process the same directory twice. Once I have the hash for a subtree I add it as a 040000 mode entry in the parent tree. After collecting all root-level entries (blobs and subtree hashes), I serialize and store the root tree object.

Screenshots

2A: `./test_tree` passing

```
(kali@kali)-[~/PES1UG24CS463-pes-vcs]
$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B: `xxd` of a tree object | `head -20`

```
(kali@kali)-[~/PES1UG24CS463-pes-vcs]
$ find .pes/objects -type f
.pes/objects/68/759ce4a0cf254ae151ecf005226718d158efa5fadd1c91cd7d92810ed54b48
.pes/objects/a4/fb87a93ce3822620d5299524f3ca44c8a11ff3dbb28f0dfd14b13c3fd99aa9
.pes/objects/2d/67115baa415eefe7dd6f7df2cd6b9fa17024aa08a5bdf2e78f9278549e1fad

(kali@kali)-[~/PES1UG24CS463-pes-vcs]
$ xxd .pes/objects/2d/67115baa415eefe7dd6f7df2cd6b9fa17024aa08a5bdf2e78f9278549e1fad | head -5
00000000: 7472 6565 2035 3200 3130 3036 3434 2074  tree 52.100644 t
00000010: 6573 7466 696c 652e 7478 7400 6875 9ce4  estfile.txt.hu..
00000020: a0cf 254a e151 ecf0 0522 6718 d158 efa5  ..%J.Q... "g..X..
00000030: fadd 1c91 cd7d 9281 0ed5 4b48          .....}....KH
```

Phase 3: The Index (Staging Area)

Implementation

The index file at `.pes/index` tracks what's staged for the next commit. Each line has the format: mode hash mtime size path. I implemented three functions for this.

`index_load` reads the file line by line with `sscanf`. If the file doesn't exist yet it's not an error — we just return an empty index. That case happens on a fresh repo before anything has been staged.

`index_save` writes everything back out. I sort entries by path first before writing. The write goes to `.pes/index.tmp`, I call `fsync` on it, then rename it to `.pes/index`. Again using the atomic rename to avoid corruption if something goes wrong mid-write.

`index_add` is the actual staging function. It reads the file from disk, stores a blob in the object store, then either updates the existing entry for that path or adds a new one. I use `stat()` to record the mtime and size so we can later detect if the file changed on disk after being staged.

Screenshots

3A: *pes init* → *pes add* → *pes status* output

```
(kali@kali)-[~/PES1UG24CS463-pes-vcs]
$ ./pes init
echo "Hello PES-VCS" > file1.txt
echo "Version Control is fun" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
    staged:   file1.txt
    staged:   file2.txt
    staged:   testfile.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    untracked: Makefile
    untracked: commit.c
    untracked: tree.c
    untracked: pes.h
    untracked: index.h
    untracked: pes.c
    untracked: test_sequence.sh
    untracked: index.c
    untracked: commit.h
    untracked: object.c
    untracked: test_tree
    untracked: test_tree.c
    untracked: test_objects.c
    untracked: tree.h
    untracked: test_objects
```

3B: *cat .pes/index*

```
(kali@kali)-[~/PES1UG24CS463-pes-vcs]
$ cat .pes/index
100644 c58493426d45950cfb24623388a0f9545cbb33905e58edad802c0f1dd1192901 1776448115 14 file1.txt
100644 a9450fa806617dfa8fa10a0387d3a910702ba3bebf40dea61277aa194f229b74 1776448115 23 file2.txt
100644 68759ce4a0cf254ae151ecf005226718d158efa5fadd1c91cd7d92810ed54b48 1776447709 14 testfile.txt
```

Phase 4: Commits and History

Implementation

`commit_create` is the main function here. When you run `pes commit` it needs to snapshot whatever is staged, write a commit object, and update HEAD.

The steps: call `tree_from_index` on the current index to get the root tree hash. Read the current HEAD to get the parent commit hash — on the first commit there's no parent so that field stays empty. Fill in the `Commit` struct with the tree hash, parent, author from `pes_author()`, and current Unix timestamp. Serialize it, write it with `object_write`, then call `head_update` to point `.pes/refs/heads/main` at the new commit hash.

I read through `commit_parse` and `commit_serialize` before writing this to understand the text format the commit is stored in. It's basically: tree line, parent line (optional), author line, blank line, then the message.

Screenshots

4A: ./pes log with three commits

```
(kali㉿kali)-[~/PES1UG24CS463-pes-vcs]
$ # First commit
./pes commit -m "Initial commit"

# Second commit
echo "More data" >> file1.txt
./pes add file1.txt
./pes commit -m "Update file1"

# Third commit
echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Committed: 626ba727fbd3 ... Initial commit
Committed: 95047f495cf4 ... Update file1
Committed: edd6f92105e0 ... Add farewell
commit edd6f92105e061b9c7c0ede887469d1704bdf1efef86638e1e5b0e9d462dee1a
Author: Soumik Sinha <PES1UG24CS463>
Date: 1776448192

    Add farewell

commit 95047f495cf442594d8b4a1c7ab66e2d3eb11c6b38e28463618ec2fdcd6010c3
Author: Soumik Sinha <PES1UG24CS463>
Date: 1776448192

    Update file1

commit 626ba727fbd3bf30414f7007a3a683c8a52fd1733e3f3164981fce2496ec825c
Author: Soumik Sinha <PES1UG24CS463>
Date: 1776448192

    Initial commit

commit a4fb87a93ce3822620d5299524f3ca44c8a11ff3dbb28f0dfd14b13c3fd99aa9
Author: Soumik Sinha <PES1UG24CS463>
Date: 1776447715

    Generate tree for screenshot
```

4B: `find .pes -type f | sort` after three commits

```
(kali㉿kali)-[~/PES1UG24CS463-pes-vcs]
$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2d/67115baa415eefe7dd6f7df2cd6b9fa17024aa08a5bdf2e78f9278549e1fad
.pes/objects/3f/c2d57aa9162ceb15bbbf13f059ac8d67902aa76fd27741cd3d759dcace90c9
.pes/objects/62/6ba727fbd3bf30414f7007a3a683c8a52fd1733e3f3164981fce2496ec825c
.pes/objects/68/759ce4a0cf254ae151ecf005226718d158efa5fadd1c91cd7d92810ed54b48
.pes/objects/8f/1a3049c75839dd00e4529515161b1f12013bc1b7dcacb2f5705bdcc95a688f
.pes/objects/95/047f495cf442594d8b4a1c7ab66e2d3eb11c6b38e28463618ec2fcd6010c3
.pes/objects/a3/34cd2db0d2a5e33040049a929e4e2c2e96256041a7a4e6c0c3bf7314a61ee1
.pes/objects/a4/fb87a93ce3822620d5299524f3ca44c8a11ff3dbb28f0dfd14b13c3fd99aa9
.pes/objects/a9/450fa806617dfa8fa10a0387d3a910702ba3bebf40dea61277aa194f229b74
.pes/objects/b3/8e00928154b0d258bca6ef57ea5dc269b9ab977de3c38c3ee8a51bc77a385d
.pes/objects/c5/8493426d45950cfb24623388a0f9545cbb33905e58edad802c0f1dd1192901
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ed/d6f92105e061b9c7c0ede887469d1704bdf1efef86638e1e5b0e9d462dee1a
.pes/refs/heads/main
```

4C: `cat .pes/refs/heads/main` and `cat .pes/HEAD`

```
(kali㉿kali)-[~/PES1UG24CS463-pes-vcs]
$ cat .pes/refs/heads/main
edd6f92105e061b9c7c0ede887469d1704bdf1efef86638e1e5b0e9d462dee1a

(kali㉿kali)-[~/PES1UG24CS463-pes-vcs]
$ cat .pes/HEAD
ref: refs/heads/main
```

Integration test: make test-integration passing

```
Initial commit

— Second Commit —
Committed: 53cc325fac85 ... Update file.txt

— Third Commit —
Committed: 005cd706cbb6 ... Add farewell

— Full History —
commit 005cd706cbb6d943dd52750ec73f4f97845a0ef9985a3d46c7fcf9447516226a
Author: Soumik Sinha <PES1UG24CS463>
Date: 1776448321

Add farewell

commit 53cc325fac853fb2e0213aad3075464eb60d1703819034eb26aec80ca27269d3
Author: Soumik Sinha <PES1UG24CS463>
Date: 1776448321

Update file.txt

commit df2b9c1f2068924a487e30fc242fe0f1f24c6bc5b75f84791ca8cd171390f90e
Author: Soumik Sinha <PES1UG24CS463>
Date: 1776448321

Initial commit

— Reference Chain —
HEAD:
ref: refs/heads/main
refs/heads/main:
005cd706cbb6d943dd52750ec73f4f97845a0ef9985a3d46c7fcf9447516226a

— Object Store —
Objects created:
10
.pes/objects/00/5cd706cbb6d943dd52750ec73f4f97845a0ef9985a3d46c7fcf9447516226a
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/53/cc325fac853fb2e0213aad3075464eb60d1703819034eb26aec80ca27269d3
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafaaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/df/2b9c1f2068924a487e30fc242fe0f1f24c6bc5b75f84791ca8cd171390f90e

≡ All integration tests completed ≡
```

Phase 5: Branching and Checkout (Analysis)

Q5.1 — How would you implement pes checkout?

A branch is just a file. `.pes/refs/heads/main` contains one line with the commit hash. Creating a branch is just creating a new file in that same directory.

To implement pes checkout `<branch>`:

- Read `.pes/refs/heads/<branch>` to get the target commit hash. Error out if it doesn't exist.
- Parse the commit to get the root tree hash.
- Walk the tree recursively. For every blob entry, read it from the object store and write the file to disk. Create directories as needed.
- Compare the current HEAD tree to the target tree — any path that exists now but not in the target needs to be deleted from the working directory.
- Write ref: `refs/heads/<branch>` into `.pes/HEAD`.
- Update the index to match the checked-out tree.

What makes this complicated is that you have to handle deletions, additions, and modifications all together. If it fails halfway the working directory is in a broken state. Real git does a dry-run check before touching anything.

Q5.2 — Detecting dirty working directory conflicts

Before checkout does anything it should check whether it's safe to overwrite files. The way to do it: for each file in the current index, check if the mtime or size on disk differs from what the index recorded. If yes, hash the current file contents and compare to the index hash to confirm it changed. If it did change, mark it as locally modified.

Then walk the target branch's tree and find files that have a different blob hash compared to the current branch. The intersection of these two sets is the conflict — files that are both locally modified and different in the target branch. For those, checkout has to refuse. Files that are different between branches but unchanged locally are fine to overwrite.

Q5.3 — Detached HEAD and recovery

Detached HEAD means `.pes/HEAD` contains a raw commit hash instead of ref: `refs/heads/main`. This happens when you check out a specific commit directly.

If you commit in this state, each new commit is written to the object store and HEAD is updated to the new hash, but no branch file gets touched. The commits exist on disk but are only reachable through HEAD. The moment you switch to a branch, HEAD points elsewhere and those commits become orphaned — no reference points to them anymore.

To recover: if you still have the hash somewhere (terminal history, etc.) you can manually write it into a new branch file — `echo <hash> > .pes/refs/heads/recovered`. If the hash is completely

lost you'd have to scan through all object files looking for commit-type objects that aren't reachable from any branch, similar to what git fsck does.

Phase 6: Garbage Collection (Analysis)

Q6.1 — Algorithm to find unreachable objects

This is a mark-and-sweep. Two phases:

Mark phase: go through every file in `.pes/refs/heads/` to get all branch tip hashes. For each commit hash, add it to a reachable set, then parse the commit to get its tree hash and parent hash — add those too. Walk the tree recursively adding every blob and subtree hash. Then follow the parent pointer to the previous commit and repeat until the first commit. Do this for every branch.

Sweep phase: walk every file under `.pes/objects/`. Reconstruct the hash from the directory and filename. If it's not in the reachable set, delete it.

The reachable set should be a hash set so lookup is $O(1)$. For 100,000 commits and 50 branches, with maybe 100 objects per commit and significant deduplication across history, you might visit a few million objects in the mark phase and scan the same number of files in the sweep phase. That's manageable.

Q6.2 — GC race condition with concurrent commits

The race condition goes like this:

- GC finishes the mark phase and has its reachable set.
- A concurrent commit runs and writes a new blob B to the object store. B is on disk but nothing references it yet — the tree and commit haven't been written.
- GC's sweep phase runs. B wasn't in the mark phase so it looks unreachable. GC deletes it.
- The commit finishes writing its tree and commit objects, which reference B. Now the repo has a dangling reference to a deleted object — corruption.

How git avoids this: the main protection is a grace period. `git gc` by default never deletes objects newer than two weeks, regardless of reachability. Any in-progress commit will complete in seconds so it's safe. Git also uses lock files when updating refs, and the reflog keeps recent commits reachable even without a branch pointing to them, so recent orphaned commits don't get collected too early.